

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

06 - Quicksort

Prof. Alex Kutzke

Junho de 2021

Tradução adaptada dos slides presentes em [1]. Imagens e exemplos também retirados do mesmo material.

[1] <<https://algs4.cs.princeton.edu/lectures/keynote/23Quicksort.pdf>>

Roteiro

- Ideia e contexto do Quicksort;
- Particionamento simples;
- Implementação;
- Análise do Quicksort;
- Otimizações práticas;

Mergesort e Quicksort

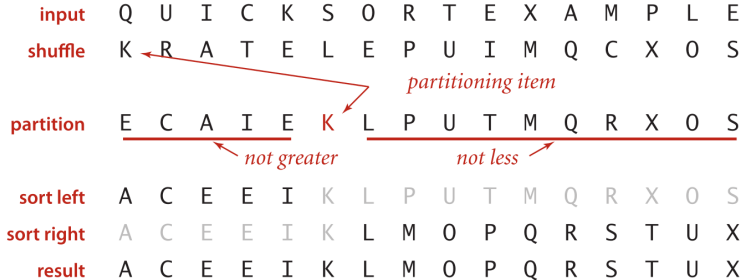
- Dois algoritmos clássicos de ordenação;
- O entendimento científico de suas propriedades permite o desenvolvimento de sistemas de ordenação com aplicações práticas;
- Quicksort está entre os 10 algoritmos mais importantes do Século XX;
- São algoritmos amplamente utilizados em aplicações do nosso dia-a-dia;

Ideia Básica

1. Rearranjar array¹;
2. Particionar o array de tal forma que, para algum j :
 - Posição $a[j]$ está em sua posição final;
 - Nenhum elemento maior do que $a[j]$ está à esquerda de $a[j]$;
 - Nenhum elemento menor do que $a[j]$ está à direita de $a[j]$;
3. Ordenar recursivamente os subarrays (de 0 até $j - 1$ e de $j + 1$ até $n - 1$);

¹Veremos em breve a importância desse passo

Ideia Básica



Fonte: (SEdgeWICK; WAYNE, 2014)

Tony Hoare

- Inventou Quicksort para traduzir Russo para Inglês;
- Aprendeu Algol 60 (e recursão);
- Implementou Quicksort;

Algorithms

```
ALGORITHM 64
QUICKSORT
C. A. R. HOARE
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

procedure quicksort (A,M,N); value M,N;
array A; integer M,N;
comment Quicksort is a very fast and convenient method of
sorting an array in the random-access store of a computer. The
entire contents of the store may be sorted, since no extra space is
required. The average number of comparisons made is  $2(M-N) \ln$ 
 $(N-M)$ , and the average number of exchanges is one sixth this
amount. Suitable refinements of this method will be desirable for
its implementation on any actual computer;
begin      integer I,J;
            if  $M < N$  then begin partition (A,M,N,I,J);
                quicksort (A,M,J);
                quicksort (A, I, N)
            end
end      quicksort
```



Tony Hoare - 1980 Turing Award

Communications of the ACM
(Julho 1961)

Tony Hoare



Tony Hoare - 1980 Turing Award

“Existem duas formas para construir um software: uma delas é criá-lo tão simples de forma que, obviamente, não existam deficiências, e a outra é criá-lo tão complexo de forma que não existam deficiências óbvias. O primeiro modo é muito mais difícil.”

Tony Hoare



Tony Hoare - 1980 Turing Award

"Eu chamo de "meu erro de um bilhão de dólares". Foi a invenção do "Null reference"(para Algol) em 1965... Isso levou incontáveis erros, vulnerabilidades e crashes de sistemas, o que provavelmente causou bilhões de dólares em danos e dores nos últimos quarenta anos."

Bob Sedgewick

- Refinou e popularizou o quicksort;
- Analisou várias versões do algoritmo;



Bob Sedgewick

Programming
Techniques

S. L. Graham, R. L. Rivest
Editors

Implementing Quicksort Programs

Robert Sedgewick
Brown University

This paper is a practical study of how to implement the Quicksort sorting algorithm and its best variants on real computers, including how to apply various code optimization techniques. A detailed implementation combining the most effective improvements to Quicksort is given, along with a discussion of how to implement it in assembly language. Analytic results describing the performance of the programs are summarized. A variety of special situations are considered from a practical standpoint to illustrate Quicksort's wide applicability as an internal sorting method which requires negligible extra storage.

Key Words and Phrases: Quicksort, analysis of algorithms, code optimization, sorting
CR Categories: 4.0, 4.6, 5.25, 5.31, 5.5

Acta Informatica 7, 327—355 (1977)
© by Springer-Verlag 1977

The Analysis of Quicksort Programs*

Robert Sedgewick

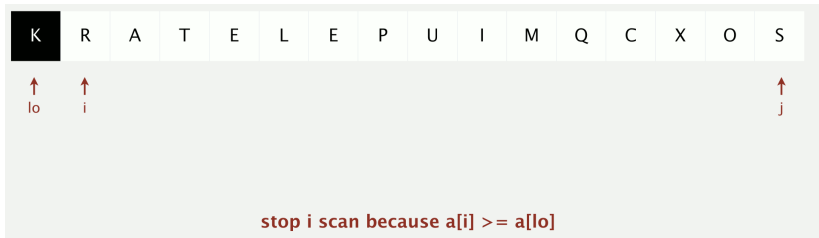
Received January 19, 1976

Summary. The Quicksort sorting algorithm and its best variants are presented and analyzed. Results are derived which make it possible to obtain exact formulas describing the total expected running time of particular implementations on real computers of Quicksort and an improvement called the median-of-three modification. Detailed analysis of the effect of an implementation technique called loop unwrapping is presented. The paper is intended not only to present results of direct practical utility, but also to illustrate the intriguing mathematics which arises in the complete analysis of this important algorithm.

Particionamento Simples

Repita até que i e j se cruzem:

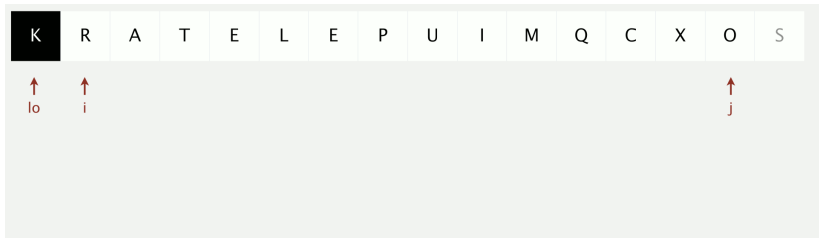
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

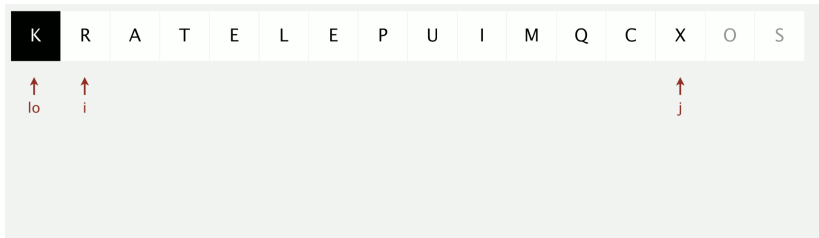
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

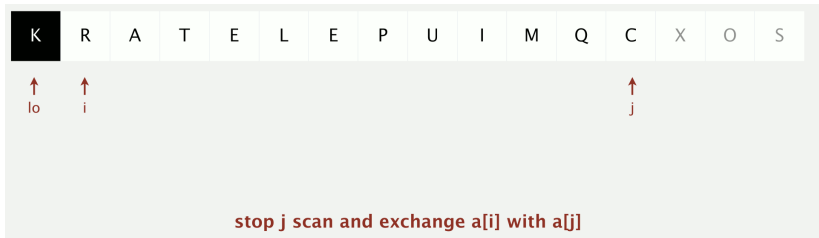
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

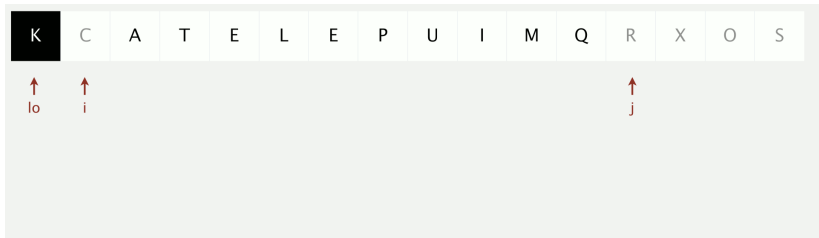
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

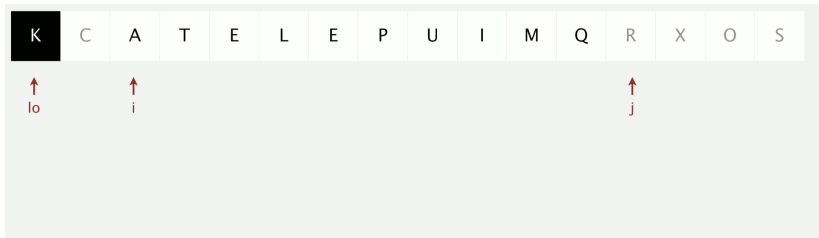
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

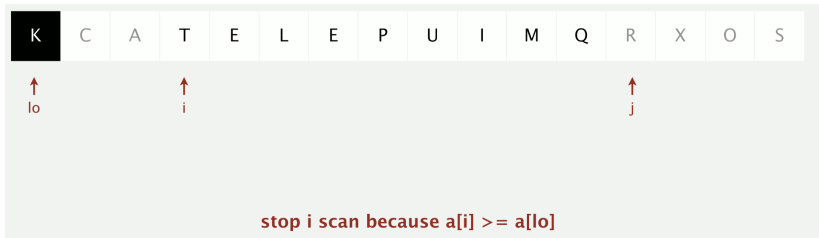
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

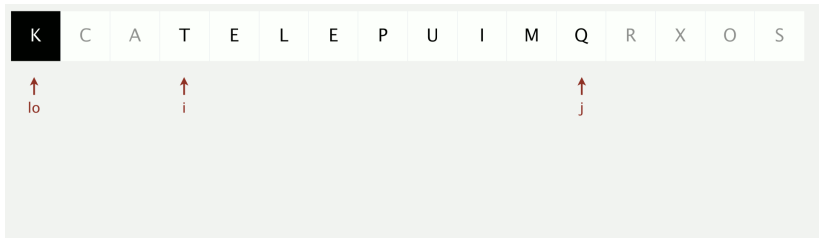
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

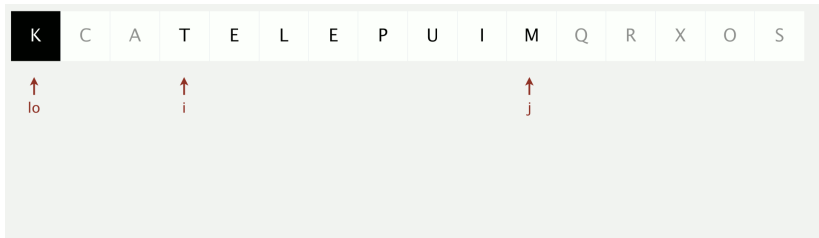
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

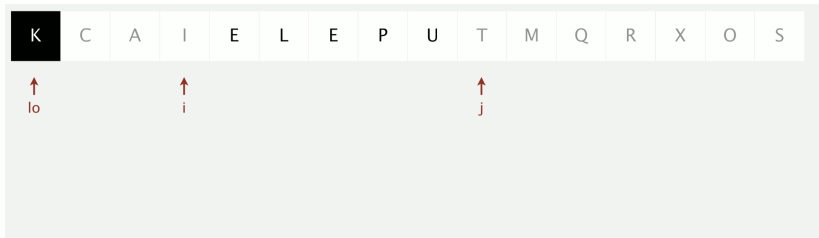
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

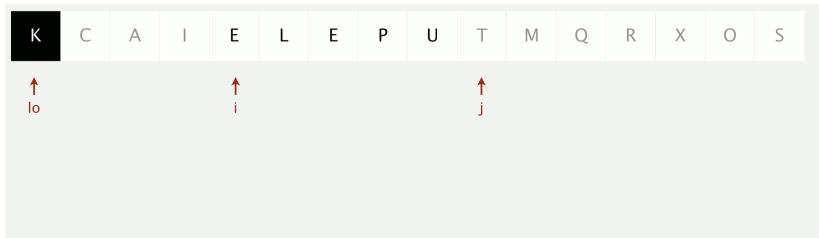
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

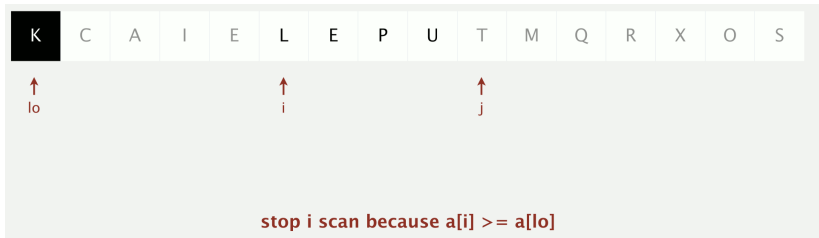
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

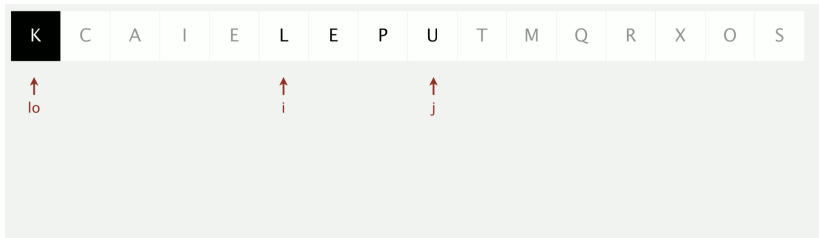
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

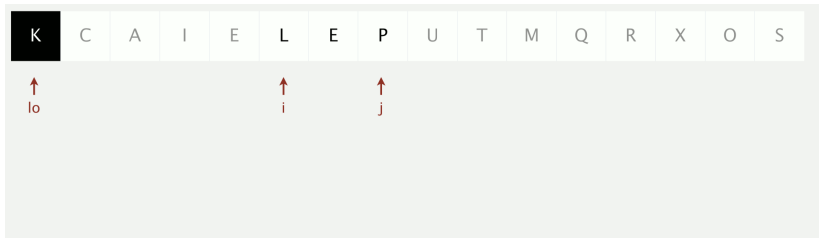
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

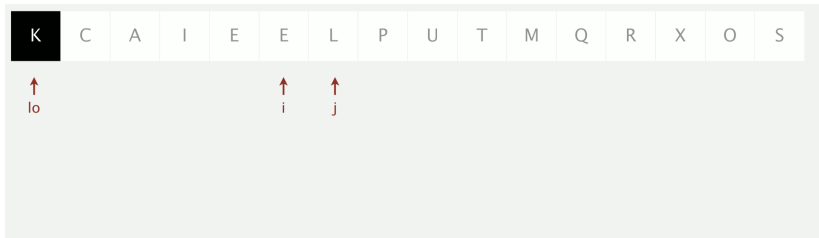
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

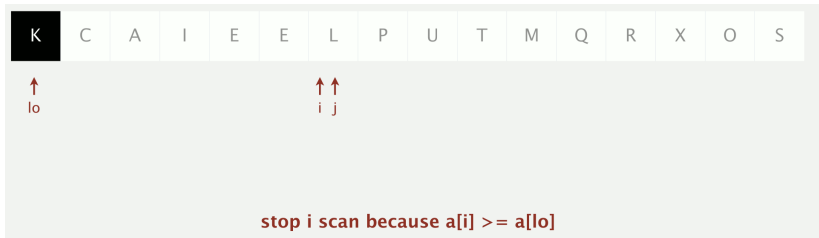
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

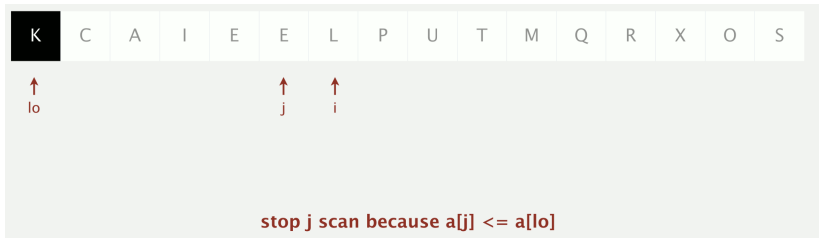
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

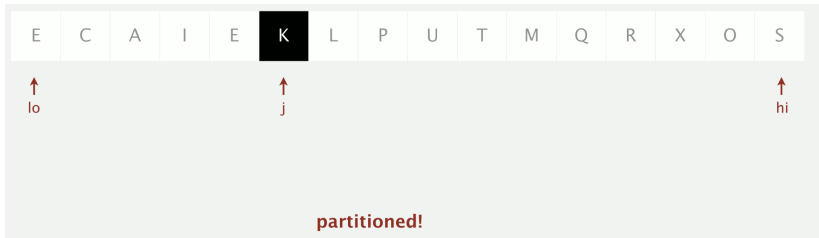
- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Particionamento Simples

Repita até que i e j se cruzem:

- Varrer i da esquerda para a direita enquanto $a[i] < a[lo]$;
- Varrer j da direita para a esquerda enquanto $a[j] > a[lo]$;
- Troque $a[i]$ com $a[j]$.



Implementação Particionamento Simples

```
1  int partition(int * a, int lo, int hi)
2  {
3      int i = lo, j = hi+1;
4      while(1)
5      {
6          while(a[++i] < a[lo]) // Encontra item na esquerda para a troca
7              if(i == hi) break;
8
9          while(a[lo] < a[--j]) // Encontra item na direita para a troca
10             if(j == lo) break;
11
12             if(i >= j) break; // Checa se ponteiros se cruzam
13             swap(a, i, j); // Troca
14     }
15
16     swap(a, lo, j); // Troca com o pivo
17     return(j); // Retorna o índice do elemento na
18 } // sua posição "ordenada"
```

Particionamento Simples



Implementação Quicksort

```
1  int partition(int * a, int lo, int hi);
2
3  void quicksort_(int * a, int lo, int hi)
4  {
5      if(hi <= lo) return;
6      int j = partition(a, lo, hi);
7      quicksort_(a, lo, j-1);
8      quicksort_(a, j+1, hi);
9  }
10
11 void quicksort(int * a, int n)
12 {
13     shuffle(a,n);           // Veremos a seguir a importância do shuffle
14     quicksort_(a,0,n-1);
15 }
```

Execução Quicksort

	lo	j	hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
initial values				Q	U	I	C	K	S	O	R	T	E	X	A	M	P	L	E
random shuffle				K	R	A	T	E	L	E	P	U	I	M	Q	C	X	O	S
	0	5	15	E	C	A	I	E	K	L	P	U	T	M	Q	R	X	O	S
	0	3	4	E	C	A	E	I	K	L	P	U	T	M	Q	R	X	O	S
	0	2	2	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
	0	0	1	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
	1		1	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
	4		4	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
	6	6	15	A	C	E	E	I	K	L	P	U	T	M	Q	R	X	O	S
	7	9	15	A	C	E	E	I	K	L	M	O	P	T	Q	R	X	U	S
	7	7	8	A	C	E	E	I	K	L	M	O	P	T	Q	R	X	U	S
	8		8	A	C	E	E	I	K	L	M	O	P	T	Q	R	X	U	S
	10	13	15	A	C	E	E	I	K	L	M	O	P	S	Q	R	T	U	X
	10	12	12	A	C	E	E	I	K	L	M	O	P	R	Q	S	T	U	X
	10	11	11	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X
	10		10	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X
	14	14	15	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X
	15		15	A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X
result				A	C	E	E	I	K	L	M	O	P	Q	R	S	T	U	X

Quicksort trace (array contents after each partition)

Detalhes de implementação

Particionamento *in-place* Utilizar um array adicional torna o particionamento mais fácil (e estável), mas não vale o custo;

Chaves iguais Quando chaves repetidas estão presentes, é melhor parar a varredura em chaves iguais a do pivô;

Manter aleatoriedade Shuffle é necessário para “garantia” de performance;

Alternativa equivalente Escolher um pivô aleatório em cada subarray;

Análise empírica (1961)

Estimativas de desempenho:

- Implementação em Algol 60;
- Computador: National Elliott 405.

Table 1

NUMBER OF ITEMS	MERGE SORT	QUICKSORT
500	2 min 8 sec	1 min 21 sec
1,000	4 min 48 sec	3 min 8 sec
1,500	8 min 15 sec*	5 min 6 sec
2,000	11 min 0 sec*	6 min 47 sec

* These figures were computed by formula, since they cannot be achieved on the 405 owing to limited store size.

Ordenação de n palavras de tamanho 6 com chaves de uma palavra.



Elliott 405 magnet disc (16k palavras)

Análise empírica

Estimativas de desempenho:

- Computador doméstico executa 10^8 comparações por segundo;
- Super computador executa 10^{12} comparações por segundo;

	insertion sort (n^2)			mergesort ($n \log n$)			quicksort ($n \log n$)		
computer	thousand	million	billion	thousand	million	billion	thousand	million	billion
home	instant	2.8 hours	317 years	instant	1 second	18 min	instant	0.6 sec	12 min
super	instant	1 second	1 week	instant	instant	instant	instant	instant	instant

Lição 1 Bons algoritmos são melhores do que super computadores.

Lição 2 Grandes algoritmos são melhores do que bons algoritmos.

Análise Quicksort

Melhor caso Número de comparações é aprox. $n \log_2 n$.

			a[]														
lo	j	hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
initial values			H	A	C	B	F	E	G	D	L	I	K	J	N	M	O
random shuffle			H	A	C	B	F	E	G	D	L	I	K	J	N	M	O
0	7	14	D	A	C	B	F	E	G	H	L	I	K	J	N	M	O
0	3	6	B	A	C	D	F	E	G	H	L	I	K	J	N	M	O
0	1	2	A	B	C	D	F	E	G	H	L	I	K	J	N	M	O
0		0	A	B	C	D	F	E	G	H	L	I	K	J	N	M	O
2		2	A	B	C	D	F	E	G	H	L	I	K	J	N	M	O
4	5	6	A	B	C	D	E	F	G	H	L	I	K	J	N	M	O
4		4	A	B	C	D	E	F	G	H	L	I	K	J	N	M	O
6		6	A	B	C	D	E	F	G	H	L	I	K	J	N	M	O
8	11	14	A	B	C	D	E	F	G	H	J	I	K	L	N	M	O
8	9	10	A	B	C	D	E	F	G	H	I	J	K	L	N	M	O
8		8	A	B	C	D	E	F	G	H	I	J	K	L	N	M	O
10		10	A	B	C	D	E	F	G	H	I	J	K	L	N	M	O
12	13	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
12		12	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
14		14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O

Análise Quicksort

Pior caso Número de comparações é aprox. $\frac{1}{2}n^2$.

			a[]														
lo	j	hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
initial values			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
random shuffle			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0	0	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	1	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
2	2	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
3	3	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
4	4	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	5	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
6	6	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
7	7	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
8	8	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
9	9	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
10	10	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
11	11	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
12	12	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
13	13	14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
14		14	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
			A	B	C	D	E	F	G	H	I	J	K	L	M	N	O

Resumo Análise Quicksort

Quicksort é um algoritmo probabilístico (*randomized*, ou *Las Vegas*):

- Corretude garantida;
- Tempo de execução depende do *shuffle* aleatório;

Caso médio Número de comparações para um array de n chaves **distintas** é $1.39n \log_2 n$ (e o número de trocas é $\frac{1}{3}n \log_2 n$):

- 39% mais comparações do que o Mergesort;
- Mais rápido do que o Mergesort na prática pois realiza menos movimentação de dados;

Melhor caso Número de comparações é $n \log_2 n$;

Pior caso Número de comparações é $\frac{1}{2}n^2$; Mas é mais provável que um raio caia no computador durante a execução

Propriedades Quicksort

Implementação apresentada (e a maioria delas) é:

- In-place;
- Não estável;

Por quê?

Propriedades Quicksort

Implementação apresentada (e a maioria delas) é:

- In-place; Particionamento utiliza espaço constante e altura da árvore de recursão é, provavelmente logarítmica
- Não estável;

Por quê?

Propriedades Quicksort

Implementação apresentada (e a maioria delas) é:

- In-place; Particionamento utiliza espaço constante e altura da árvore de recursão é, provavelmente logarítmica
- Não estável; Contra-exemplo

Por quê?

Propriedades Quicksort

Implementação apresentada (e a maioria delas) é:

- In-place; Particionamento utiliza espaço constante e altura da árvore de recursão é, provavelmente logarítmica
- Não estável; Contra-exemplo

i	j	0	1	2	3
		B ₁	C ₁	C ₂	A ₁
1	3	B ₁	C ₁	C ₂	A ₁
1	3	B ₁	A ₁	C ₂	C ₁
0	1	A ₁	B ₁	C ₂	C ₁

Insertion sort para pequenos subarrays

- Até mesmo Quicksort possui mais overhead para pequenos arrays do que Insertion sort;
- Aplicar *cutoff* (10 pode ser um bom número);

Mediana para pivô

- Melhor pivô? Mediana do array (divide a coleção exatamente ao meio);
- Utilizar a mediana exata do array (caro);
- Utilizar a mediana entre uma amostra qualquer (tamanho 3) do array (boa aproximação);
- Já evita o pior caso!

Chaves iguais

- Em aplicações práticas é comum que o conjunto de dados a ser ordenado possua algumas chaves iguais (elementos repetidos);
- Assim como no Mergesort, é possível tirar proveito de coleções desse tipo para melhorar o desempenho do Quicksort;
- Alias, a implementação vista não possui um bom desempenho para coleções com vários elementos repetidos. Consegue perceber porquê?

```
Chicago 09:25:52
Chicago 09:03:13
Chicago 09:21:05
Chicago 09:19:46
Chicago 09:19:32
Chicago 09:00:00
Chicago 09:35:21
Chicago 09:00:59
Houston 09:01:10
Houston 09:00:13
Phoenix 09:37:44
Phoenix 09:00:03
Phoenix 09:14:25
Seattle 09:10:25
Seattle 09:36:14
Seattle 09:22:43
Seattle 09:10:11
Seattle 09:22:54
```



key

Chaves iguais

Nosso algoritmo de particionamento para a varredura quando encontra elementos iguais ao pivô:



Por que não continuar a varredura quando encontra elementos iguais ao pivô?



Chaves iguais: estratégias de particionamento

Ruim Não interromper varredura em chaves iguais ao pivô
 $\sim \frac{n^2}{2}$ comparações quando todos os elementos são iguais

B	A	A	B	A	B	B	B	C	C	C
---	---	---	---	---	---	---	---	---	---	---

A	A	A	A	A	A	A	A	A	A	A
---	---	---	---	---	---	---	---	---	---	---

Bom Interromper varredura em chaves iguais ao pivô
 $\sim n \log_2 n$ comparações quando todos os elementos são iguais

B	A	A	B	A	B	C	C	B	C	B
---	---	---	---	---	---	---	---	---	---	---

A	A	A	A	A	A	A	A	A	A	A
---	---	---	---	---	---	---	---	---	---	---

Melhor Colocar todas as chaves iguais ao pivô no lugar correto. Mas como? $\sim n$ comparações quando todos os elementos são iguais

A	A	A	B	B	B	B	B	C	C	C
---	---	---	---	---	---	---	---	---	---	---

A	A	A	A	A	A	A	A	A	A	A
---	---	---	---	---	---	---	---	---	---	---

Problema da Bandeira da Holanda

Problema [Edsger Dijkstra] Dado um array de N posições e cada posição contendo uma cor (vermelha, azul ou branca), ordene o array segundo as cores da bandeira da Holanda.



Operações permitidas:

- $swap(i, j)$: trocar as cores das posições i e j ;
- $color(i)$: retorna a cor da posição i ;

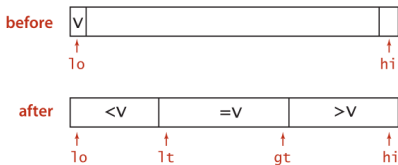
Requisitos da solução:

- Exatamente n chamadas para $color()$;
- No máximo n chamadas para $swap()$;
- Espaço extra constante.

Partição em 3 partes

Partição em três partes, tal que:

- Valores entre lt e gt são iguais ao pivô;
- Nenhum valor maior à esquerda de lt ;
- Nenhum valor menor à direita de gt ;

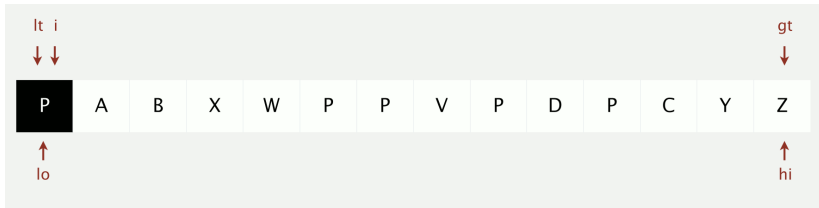


Problema da bandeira da Holanda [Edsger Dijkstra]:

- Entendimento até metade de 1990: não vale a pena resolver;
- Atualmente, Partição em 3 partes foi incorporada à biblioteca C (`qsort()`) e ao Java.

Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[lo]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[lt]$ com $a[i]$; incrementa ambos lt e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



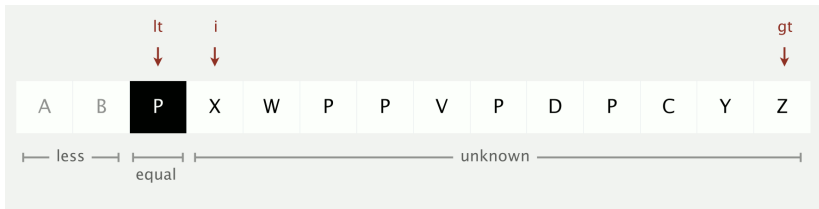
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_0]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



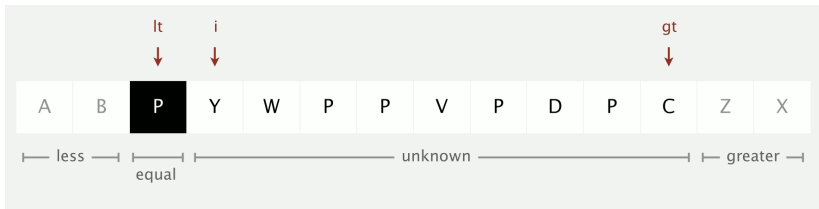
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_0]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



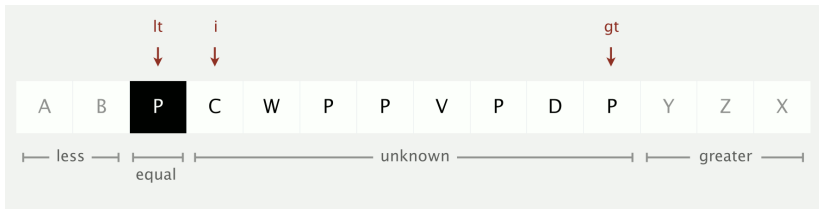
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



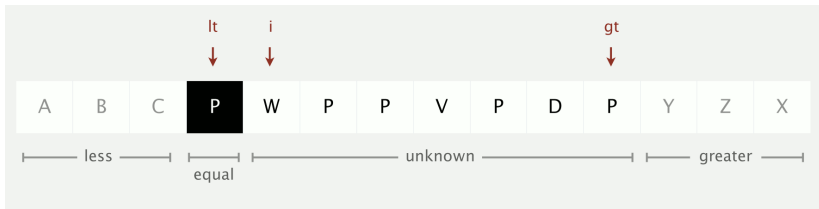
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



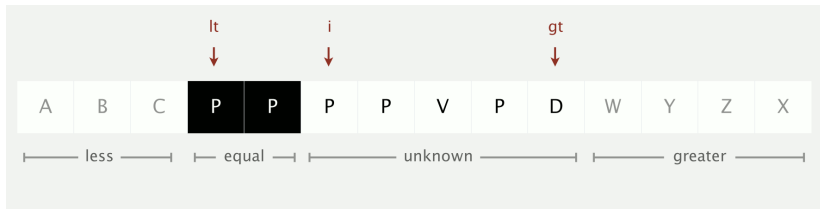
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



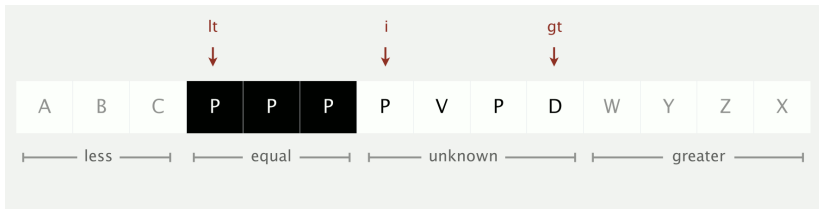
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



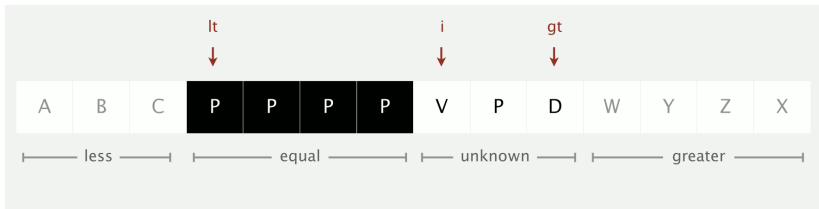
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



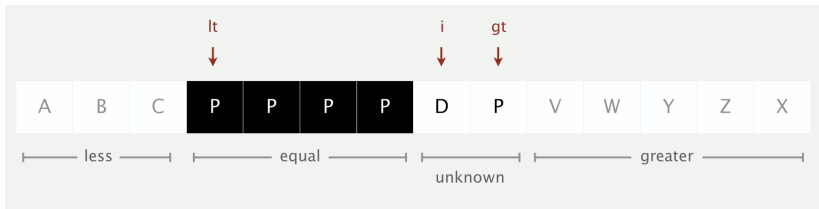
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



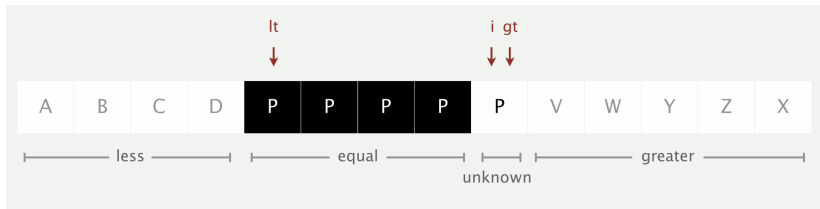
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



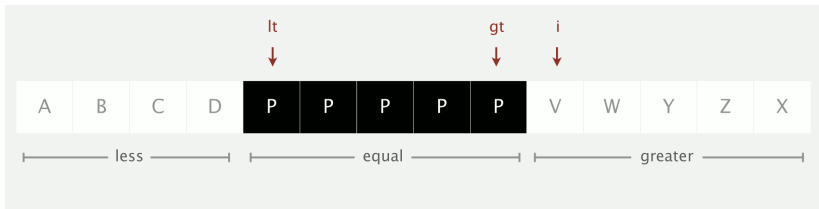
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



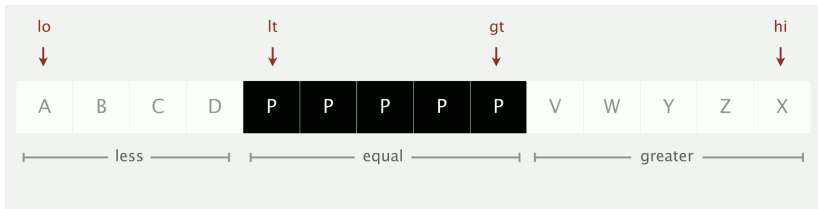
Dijkstra Particionamento em 3 partes

- Seja v o pivô, na posição $a[l_o]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[l_t]$ com $a[i]$; incrementa ambos l_t e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



Dijkstra Particionamento em 3 partes

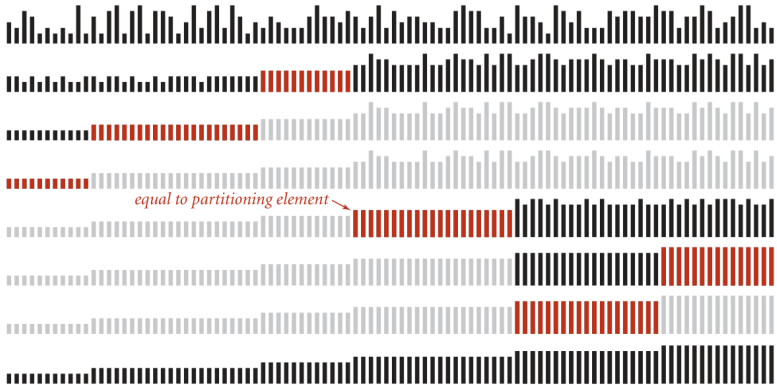
- Seja v o pivô, na posição $a[lo]$;
- Varrer da esquerda para a direita com a variável i :
 - ($a[i] < v$): troca $a[lt]$ com $a[i]$; incrementa ambos lt e i ;
 - ($a[i] > v$): troca $a[gt]$ com $a[i]$; decrementa gt ;
 - ($a[i] == v$): incrementa i ;



Implementação Particionamento em 3 partes

???

Execução 3-way Quicksort



3-way Quicksort reduz o tempo de execução de Linearítmico para linear em uma grande classe de aplicações.

Resumo Ordenação

	inplace?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	n exchanges
insertion	✓	✓	n	$\frac{1}{4} n^2$	$\frac{1}{2} n^2$	use for small n or partially ordered
shell	✓		$n \log_3 n$	?	$c n^{3/2}$	tight code; subquadratic
merge		✓	$\frac{1}{2} n \lg n$	$n \lg n$	$n \lg n$	$n \log n$ guarantee; stable
timsort		✓	n	$n \lg n$	$n \lg n$	improves mergesort when preexisting order
quick	✓		$n \lg n$	$2 n \ln n$	$\frac{1}{2} n^2$	$n \log n$ probabilistic guarantee; fastest in practice
3-way quick	✓		n	$2 n \ln n$	$\frac{1}{2} n^2$	improves quicksort when duplicate keys
?	✓	✓	n	$n \lg n$	$n \lg n$	holy sorting grail

Referências utilizadas na apresentação I

 SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.l.]: Addison-Wesley Professional, 2014.